

ACMEMobility

Progetto di Architetture Software a Microservizi, Università di Bologna, 2025–2026

Daniele D’Ugo
daniele.dugo@studio.unibo.it
Mat.: 0001186581

Daniele D’Ugo
daniele.dugo@studio.unibo.it
Mat.: 0001186581

Daniele D’Ugo
daniele.dugo@studio.unibo.it
Mat.: 0001186581

I. INTRODUZIONE

Il servizio di noleggio ACMEMobility è stato implementato come un’architettura a microservizi, tramite *docker compose*, cercando di rispettare tutti i requisiti del progetto.

II. AVVIO E UTILIZZO

L’intera architettura, comprendente tutti i servizi, con il codice nella cartella *src/* può essere avviata e gestita tramite i comandi di *docker compose*; per comodità, sono presenti due script *bash* (per sistemi Linux). *start.sh* lancia l’infrastruttura:

```
1 # dalla cartella di base del repository
2 ./src/start.sh
```

down.sh interrompe i servizi:

```
1 ./src/down.sh
```

Prima, però, è necessario scaricare la mappa (con dati di OpenStreetMap) dell’Italia nord-orientale, disponibile al link su geofabrik: <https://download.geofabrik.de/europe/italy/nord-est-latest.osm.pbf>.

Il file dovrà essere piazzato in *service-graphhopper/data/*, con nome del file *geofabrik_italy-nord-est-260517.osm.pbf*.

Da Linux, si può eseguire direttamente, per entrambi i passaggi, il comando:

```
1 # dalla cartella di base del repository
2 wget -O service-graphhopper/data/geofabrik_italy-nord-est-260517.osm.pbf \
3 https://download.geofabrik.de/europe/italy/nord-est-latest.osm.pbf
```

Il flusso delle attività parte sempre dall’utente, che in uno scenario reale sarebbe un’entità fisica e potrebbe interagire con ACMEMobility tramite un’applicazione per smartphone. Dunque, abbiamo realizzato uno script in *python* (*src/user/user.py*) che simula tutte le possibili azioni che possa compiere, in base agli argomenti utilizzati. Seguono degli esempi per ogni utilizzo. Usage:

```
1 # dalla cartella di base del repository
2 python src/user/user.py
3 # Usage: python user.py <action> <user_id> <vehicle_id> [station_id]
4 # Actions: scan, reserve, cancel, scan_reserved, park, lock, lock_assistance, steal
```

Scansione veicolo (con id *V-01*) per uso immediato, da parte dell’utente con id *USER-001*:

```
1 python src/user/user.py scan USER-001 V-01
```

Prenotazione veicolo per dopo:

```
1 python src/user/user.py reserve USER-001 V-01
```

Cancellazione della prenotazione precedente:

```
1 python src/user/user.py cancel USER-001 V-01
```

Scansione del veicolo precedentemente prenotato:

```
1 python src/user/user.py scan_reserved USER-001 V-01
```

Parcheggio del veicolo (simulazione dell'inserimento del veicolo nel dock della stazione con id *STATION-BOLOGNA-01*):

```
1 python src/user/user.py park USER-001 V-01 STATION-BOLOGNA-01
```

Richiesta di fine corsa e, dunque, blocco del veicolo precedentemente parcheggiato:

```
1 python src/user/user.py lock USER-001 V-01 STATION-BOLOGNA-01
```

Richiesta di assistenza a fine corsa, in caso di problemi con il blocco:

```
1 python src/user/user.py lock_assistance USER-001 V-01
```

Simulazione furto di un veicolo:

```
1 python src/user/user.py steal USER-001 V-01
```

Una volta avviata l'infrastruttura, è possibile accedere alla visualizzazione di Camunda al link <http://localhost:8080/camunda/app/cockpit/>, con username *demo* e password *demo*.

III. WORKFLOW E ARTEFATTI

Lo sviluppo del progetto è partito con la modellazione di una coreografia in BPMN con scopo documentativo. Usandola come schema è stato così possibile suddividere, tra i membri del gruppo, i compiti di realizzare ulteriori modelli o i singoli servizi.

La coreografia è stata poi gradualmente raffinata, fino a diventare la attuale coreografia a scopo documentativo (file *docs/diagrams/diagram-collaboration-full.bpmn*). Essa raffigura infatti tutti i partecipanti e i loro task, per rendere l'idea di quali feature debba implementare ogni servizio.

Per Camunda è stato modellato un altro diagramma di collaborazione eseguibile (file *src/camunda-resources/diagram-collaboration-camunda.bpmn*), con la principale differenza che non implementa nel dettaglio tutti i partecipanti, ma solo il servizio principale di ACMEMobility, che era richiesto che fosse gestito tramite BPMS.

Successivamente, sono stati realizzati un diagramma di documentazione della SOA in UML, con profilo TinySOA (file *docs/diagrams/diagram-SOA.uml* e *docs/diagrams/diagram-SOA-UML.uml*), e una coreografia con sintassi formale, necessaria per le proiezioni della coreografia sui ruoli e per l'analisi di *connectedness*.

I servizi implementati, invece, sono:

- ACME (*src/service-acme-worker/*): il worker che implementa i service task del BPMS per il core di ACMEMobility, scritto in *python* con l'apposito pacchetto per Camunda;
- bank (*src/service-bank/*): la banca, in *jolie*;
- Fleet Management (*src/service-fleet-management/*): implementato come un router, in *Nginx*, così che appaia esso stesso come un'architettura a microservizi;
 - Fleet Management Battery (*src/service-fleet-battery/*): microservizio per gestire la batteria dei veicoli, in *python* con *Flask*;
 - Fleet Management Tracking (*src/service-fleet-tracking/*): microservizio per gestire posizione e informazioni aggiuntive dei veicoli, in *python* con *Flask*;
- grapphopper (*src/service-graphhopper/*): routing service per fornire dei percorsi reali ai veicoli, date posizione di partenza e destinazione, nonché per visualizzarli in tempo reale; configurato con il container *docker* già esistente;
- station (*src/service-stations/*): implementazione di una singola stazione, in *python* con *Flask*;
- vehicle (*src/service-vehicle/*): implementazione di un singolo veicolo, in *python* con *Flask*.

Tutti i servizi sono gestiti tramite un'architettura con *docker-compose*, che si occupa anche di istanziare molteplici veicoli e stazioni, con la stessa immagine e implementazione. Il file è scritto come template in

src/docker-compose.template.yml; uno script *bash* (*src/bash-scripts/compile-dockercompose.sh*, già invocato da *start.sh*) lo lavora, per esempio sostituendo delle variabili con quelle in *src/.env*, generando quello finale (*src/docker-compose.generated.yml*). In questo modo, i veicoli e le stazioni istanziati sono ben riconoscibili poiché generati con identificativi definiti tramite variabili di ambiente.

Inoltre, tutti i servizi usano *API REST* e comunicano tramite *http*, ad eccezione della banca, che usa *SOAP* come richiesto.

Per la coreografia in Camunda, come già accennato, tutti i service task sono implementati tramite un worker in python. Solo delle operazioni banali, principalmente le condizioni di scelta per i gateway esclusivi, sono implementate come espressioni *inline*, con sintassi *JUEL* (*Java Unified Expression Language*), utilizzando delle variabili globali che vengono configurate dallo stesso worker. L'implementazione esterna, infatti, fornisce grandi vantaggi nello sviluppo, in termini di gestibilità, mantenibilità e leggibilità del codice, possibilità di debugging e gestione coerente e ordinata dei file.

In *docs/implementation/* sono presenti le documentazioni sia per le API dei servizi, sia per le variabili globali di Camunda (*bpms-variables.md*).

IV. IMPLEMENTAZIONE

In questa sezione presentiamo il modo in cui vengono eseguiti i processi (e quindi la coreografia), insieme a commenti e motivazioni per le scelte di design.

A. La coreografia

Come già accennato, il primo processo a partire è quello dell'utente, che va poi ad avviare a catena gli altri, con dei *message start event*. Nella **fase 1**, la **prenotazione**, l'utente parte con una scelta: scansionare il codice QR di un veicolo subito oppure prenotarne uno; nel secondo caso, può in seguito scansionare il veicolo per far partire la corsa, oppure cancellare la prenotazione e quindi terminare il suo processo. In entrambi i casi, dopo essere giunto con successo alla scansione, inizia la **fase 2**, il **tragitto**, estremamente semplice dal punto di vista dell'utente, meno dal punto di vista di ACMEMobility.

Infine, nella **fase 3**, la **conclusione**, l'utente deve interrompere la corsa e attendere la notifica di conferma da parte dell'applicazione. L'interruzione della corsa è in realtà ulteriormente articolata: bisogna prima inserire il veicolo in un terminale della stazione (chiamato anche "dock") e, successivamente, richiedere il blocco del veicolo ad ACMEMobility, tramite l'applicazione. In caso di problemi nel parcheggiare, si possono ripetere più volte le due operazioni di parcheggio e blocco finché non si abbia successo, oppure contattare l'assistenza (interrompendo il processo della coreografia): in questo modo, un addetto verrà inviato a controllare personalmente il veicolo, mentre per l'utente ACMEMobility procederà con l'addebito, in modo che possa andarsene e non debba rimanere ad aspettare. Qui, per gestire il fatto che il veicolo venga lasciato incustodito, assumiamo che, nel mondo reale, esso abbia qualche sistema di bloccaggio automatico della guida, o che magari funzioni solo quando un utente correttamente autorizzato (attraverso la **fase 1**) sia nelle vicinanze (il che potrebbe essere confermato dalla vicinanza del suo telefono, per esempio via *Bluetooth*).

Nel parcheggio, mentre il blocco del veicolo passa attraverso ACMEMobility (attraverso un *message intermediate throw event*), il precedente inserimento è modellato direttamente come un messaggio verso la stazione: nel mondo reale, infatti, immaginiamo che veicolo e terminale possano essere configurati con sensori per comunicazione a breve raggio (per esempio, per riconoscere un tag *RFID* o *NFC* sul veicolo), grazie ai quali la stazione può identificare se ci sia un veicolo inserito, prima di bloccarlo. D'altra parte, ACME, all'arrivo della notifica di parcheggio, procede, in ordine, a chiedere alla stazione il bloccaggio, effettuare l'addebito di fine corsa e confermare l'operazione all'utente con una notifica. La separazione in inserimento e bloccaggio è stata fatta per rendere più solido il design di questa parte: in questo modo, l'applicazione dell'utente deve solo comunicare gli identificativo di stazione e veicolo, e la stazione saprà automaticamente quale terminale bloccare; la comunicazione a breve raggio, inoltre, permette al veicolo e

stazione di scambiare eventuali informazioni di conferma.

V. RISULTATI

VI. CONCLUSIONE E DISCUSSIONE FINALE

REFERENCES